i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$T[i]$	b	i	p	p	i	t	y	b	o	p	p	i	t	y	b	o	o	∅	∅
r_i	1	3	□	8	4	□	12	2	□	9	7	□	10	11	□	6	5	0	0

Figure 32.16 The ranks r_1 through r_{n+3} for the text $T =$ `bippityboppityboo` with $n = 17$.

G. Extending the text T by the two special characters ∅∅, so that T now has $n + 2$ characters, consider each suffix $T[i:]$ for $i = 1, 2, \ldots, n + 2$. Assign a rank r_i to each suffix $T[i:]$. For the two special characters ∅∅, set $r_{n+1} = r_{n+2} = 0$. For the sample positions of T, base the rank on the list of sorted sample positions of T. The rank is currently undefined for the nonsample positions of T. For these positions, set $r_i = \square$. Figure 32.16 shows the ranks for $T =$ `bippityboppityboo` with $n = 17$.

H. Sort the nonsample suffixes by comparing tuples $(T[i], r_{i+1})$. In our example, we get $T[15:] < T[12\ \ :] < T[9\ \ :] < T[3\ \ :] < T[6\ \ :]$ because $(\texttt{b}, 6) < (\texttt{i}, 10) < (\texttt{o}, 9) < (\texttt{p}, 8) < (\texttt{t}, 12)$.

3. Merge the sorted sets of suffixes. From the sorted set of suffixes, determine the suffix array of T.

This completes the description of a linear-time algorithm for computing suffix arrays. The following parts of this problem ask you to show that certain steps of this algorithm are correct and to analyze the algorithm's running time.

a. Define a ***nonempty suffix*** at position i of the text P created in substep B as all metacharacters from position i of P up to and including the first metacharacter of P in which ∅ appears or the end of P. In the example shown in Figure 32.15, the nonempty suffixes of P starting at positions 1, 4, and 11 of P are (`bip`) (`pit`) (`ybo`) (`ppi`) (`tyb`) (`oo`∅), (`ppi`) (`tyb`) (`oo`∅), and (`ybo`) (`o`∅∅), respectively. Prove that the order of suffixes of P is the same as the order of its nonempty suffixes. Conclude that the order of suffixes of P gives the order of the sample suffixes of T. (*Hint:* If P contains duplicate metacharacters, consider separately the cases in which two suffixes both start in P_1, both start in P_2, and one starts in P_1 and the other starts in P_2. Use the property that ∅ appears in the last metacharacter of P_1.)

b. Show how to perform substep C in $\Theta(n)$ time, bearing in mind that in a recursive call, the characters in T are actually ranks in P' in the caller.

c. Argue that the tuples in substep H are unique. Then show how to perform this substep in $\Theta(n)$ time.

d. Consider two suffixes $T[i:]$ and $T[j:]$, where $T[i:]$ is a sample suffix and $T[j:]$ is a nonsample suffix. Show how to determine in $\Theta(1)$ time whether $T[i:]$ is lexicographically smaller than $T[j:]$. (*Hint:* Consider separately the cases in which $i \bmod 3 = 1$ and $i \bmod 3 = 2$. Compare tuples whose elements are characters in T and ranks as shown in Figure 32.16. The number of elements per tuple may depend on whether $i \bmod 3$ equals 1 or 2.) Conclude that step 3 can be performed in $\Theta(n)$ time.

e. Justify the recurrence $T(n) \le T(2n/3 + 2) + \Theta(n)$ for the running time of the full algorithm, and show that its solution is $O(n)$. Conclude that the algorithm runs in $\Theta(n)$ time.

32-3 Burrows-Wheeler transform

The ***Burrows-Wheeler transform***, or ***BWT***, for a text T is defined as follows. First, append a new character that compares as lexicographically less than every character of T, and denote this character by `$` and the resulting string by T'. Letting n be the length of T', create n rows of characters, where each row is one of the n cyclic rotations of T'. Next, sort the rows lexicographically. The BWT is then the string of n characters in the rightmost column, read top to bottom.

For example, let $T =$ `rutabaga`, so that $T' =$ `rutabaga$`. The cyclic rotations are

```
rutabaga$
utabaga$r
tabaga$ru
abaga$rut
baga$ruta
aga$rutab
ga$rutaba
a$rutabag
$rutabaga
```

Sorting the rows and numbering the sorted rows gives

```
1  $rutabaga
2  a$rutabag
3  abaga$rut
4  aga$rutab
5  baga$ruta
6  ga$rutaba
7  rutabaga$
8  tabaga$ru
9  utabaga$r
```

The BWT is the rightmost column, `agtbaa$ur`. (The row numbering will be helpful in understanding how to compute the inverse BWT.)

The BWT has applications in bioinformatics, and it can also be a step in text compression. That is because it tends to place identical characters together, as in the BWT of `rutabaga`, which places two of the instances of `a` together. When identical characters are placed together, or even nearby, additional means of compressing become available. Following the BWT, combinations of move-to-front encoding, run-length encoding, and Huffman coding (see Section 15.3) can provide significant text compression. Compression ratios with the BWT tend improve as the text length increases.

a. Given the suffix array for T', show how to compute the BWT in $\Theta(n)$ time.

In order to decompress, the BWT must be invertible. Assuming that the alphabet size is constant, the inverse BWT can be computed in $\Theta(n)$ time from the BWT. Let's look at the BWT of `rutabaga`, denoting it by $BWT[1:n]$. Each character in the BWT has a unique lexicographic rank from 1 to n. Denote the rank of $BWT[i]$ by $rank[i]$. If a character appears multiple times in the BWT, each instance of the character has a rank 1 greater than the previous instance of the character. Here are *BWT* and *rank* for `rutabaga`:

i	1	2	3	4	5	6	7	8	9
$BWT[i]$	a	g	t	b	a	a	$	u	r
$rank[i]$	2	6	8	5	3	4	1	9	7

For example, $rank[1] = 2$ because $BWT[1] =$ `a` and the only character that precedes the first `a` lexicographically is `$` (which we defined to precede all other characters, so that `$` has rank 1). Next, we have $rank[2] = 6$ because $BWT[2] =$ `g` and five characters in the BWT precede `g` lexicographically: `$`, the three instances of `a`, and `b`. Jumping ahead to $rank[5] = 3$, that is because $BWT[5] =$ `a`, and because this `a` is the second instance of `a` in the BWT, its *rank* value is 1 greater than the *rank* value for the previous instance of `a`, in position 1.

There is enough information in *BWT* and *rank* to reconstruct T' from back to front. Suppose that you know the rank r of a character c in T'. Then c is the first character in row r of the sorted cyclic rotations. The last character in row r must be the character that precedes c in T'. But you know which character is the last character in row r, because it is $BWT[r]$. To reconstruct T' from back to front, start with `$`, which you can find in *BWT*. Then work backward using *BWT* and *rank* to reconstruct T'.

Let's see how this strategy works for `rutabaga`. The last character of T', `$`, appears in position 7 of *BWT*. Since $rank[7] = 1$, row 1 of the sorted cyclic rotations of T' begins with `$`. The character that precedes `$` in T' is the last character in row 1, which is $BWT[1]$: `a`. Now we know that the last two characters of T'

are a$. Looking up *rank*[1], it equals 2, so that row 2 of the sorted cyclic rotations of T' begins with a. The last character in row 2 precedes a in T', and that character is $BWT[2] = \texttt{g}$. Now we know that the last three characters of T' are ga$. Continuing on, we have $rank[2] = 6$, so that row 6 of the sorted cyclic rotations begins with g. The character preceding g in T' is $BWT[6] = \texttt{a}$, and so the last four characters of T' are aga$. Because $rank[6] = 4$, a begins row 4 of the sorted cyclic rotations of T'. The character preceding a in T' is the last character in row 4, $BWT[4] = \texttt{b}$, and the last five characters of T' are baga$. And so on, until all n characters of T' have been identified, from back to front.

b. Given the array $BWT[1:n]$, write pseudcode to compute the array $rank[1:n]$ in $\Theta(n)$ time, assuming that the alphabet size is constant.

c. Given the arrays $BWT[1:n]$ and $rank[1:n]$, write pseudocode to compute T' in $\Theta(n)$ time.

Chapter notes

The relation of string matching to the theory of finite automata is discussed by Aho, Hopcroft, and Ullman [5]. The Knuth-Morris-Pratt algorithm [267] was invented independently by Knuth and Pratt and by Morris, but they published their work jointly. Matiyasevich [317] earlier discovered a similar algorithm, which applied only to an alphabet with two characters and was specified for a Turing machine with a two-dimensional tape. Reingold, Urban, and Gries [377] give an alternative treatment of the Knuth-Morris-Pratt algorithm. The Rabin-Karp algorithm was proposed by Karp and Rabin [250]. Galil and Seiferas [173] give an interesting deterministic linear-time string-matching algorithm that uses only $O(1)$ space beyond that required to store the pattern and text.

The suffix-array algorithm in Section 32.5 is by Manber and Myers [312], who first proposed the notion of suffix arrays. The linear-time algorithm to compute the longest common prefix array presented here is by Kasai et al. [252]. Problem 32-2 is based on the DC3 algorithm by Kärkkäinen, Sanders, and Burkhardt [245]. For a survey of suffix-array algorithms, see the article by Puglisi, Smyth, and Turpin [370]. To learn more about the Burrows-Wheeler transform from Problem 32-3, see the articles by Burrows and Wheeler [78] and Manzini [314].

33 Machine-Learning Algorithms

Machine learning may be viewed as a subfield of artificial intelligence. Broadly speaking, artificial intelligence aims to enable computers to carry out complex perception and information-processing tasks with human-like performance. The field of AI is vast and uses many different algorithmic methods.

Machine learning is rich and fascinating, with strong ties to statistics and optimization. Technology today produces enormous amounts of data, providing myriad opportunities for machine-learning algorithms to formulate and test hypotheses about patterns within the data. These hypotheses can then be used to make predictions about the characteristics or classifications in new data. Because machine learning is particularly good with challenging tasks involving uncertainty, where observed data follows unknown rules, it has markedly transformed fields such as medicine, advertising, and speech recognition.

This chapter presents three important machine-learning algorithms: k-means clustering, multiplicative weights, and gradient descent. You can view each of these tasks as a learning problem, whereby an algorithm uses the data collected so far to produce a hypothesis that describes the regularities learned and/or makes predictions about new data. The boundaries of machine learning are imprecise and evolving—some might say that the k-means clustering algorithm should be called "data science" and not "machine learning," and gradient descent, though an immensely important algorithm for machine learning, also has a multitude of applications outside of machine learning (most notably for optimization problems).

Machine learning typically starts with a ***training phase*** followed by a ***prediction phase*** in which predictions are made about new data. For ***online learning***, the training and prediction phases are intermingled. The training phase takes as input ***training data***, where each input data point has an associated output or ***label***; the label might be a category name or some real-valued attribute. It then produces as an output one or more ***hypotheses*** about how the labels depend on the attributes of the input data points. Hypotheses can take many forms, typically some type of formula or algorithm. The learning algorithm used is often a form of gradient

descent. The prediction phase then uses the hypothesis on new data in order to make ***predictions*** regarding the labels of new data points.

The type of learning just described is known as ***supervised learning***, since it starts with a set of inputs that are each labeled. As an example, consider a machine-learning algorithm to recognize spam emails. The training data comprises a collection of emails, each of which is labeled either "spam" or "not spam." The machine-learning algorithm frames a hypothesis, possibly a rule of the form "if an email has one of a set of words, then it is likely to be spam." Or it might learn rules that assign a spam score to each word and then evaluates a document by the sum of the spam scores of its constituent words, so that a document with a total score above a certain threshold value is classified as spam. The machine-learning algorithm can then predict whether a new email is spam or not.

A second form of machine learning is ***unsupervised learning***, where the training data is unlabeled, as in the clustering problem of Section 33.1. Here the machine-learning algorithm produces hypotheses regarding the centers of groups of input data points.

A third form of machine learning (not covered further here) is ***reinforcement learning***, where the machine-learning algorithm takes actions in an environment, receives feedback for those actions from the environment, and then updates its model of the environment based on the feedback. The learner is in an environment that has some state, and the actions of the learner have an effect on that state. Reinforcement learning is a natural choice for situations such as game playing or operating a self-driving car.

Sometimes the goal in a supervised machine-learning application is not making accurate predictions of labels for new examples, but rather performing causal ***inference***: finding an explanatory model that describes how the various features of an input data point affect its associated label. Finding a model that fits a given set of training data well can be tricky. It may involve sophisticated optimization methods that need to balance between producing a hypothesis that fits the data well and producing a hypothesis that is simple.

This chapter focuses on three problem domains: finding hypotheses that group the input data points well (using a clustering algorithm), learning which predictors (experts) to rely upon for making predictions in an online learning problem (using the multiplicative-weights algorithm), and fitting a model to data (using gradient descent).

Section 33.1 considers the clustering problem: how to divide a given set of n training data points into a given number k of groups, or "clusters," based on a measure of how similar (or more accurately, how dissimilar) points are to each other. The approach is iterative, beginning with an arbitrary initial clustering and incorporating successive improvements until no further improvements occur. Clustering

is often used as an initial step when working on a machine-learning problem to discover what structure exists in the data.

Section 33.2 shows how to make online predictions quite accurately when you have a set of predictors, often called "experts," to rely on, many of which might be poor predictors, but some of which are good predictors. At first, you do not know which predictors are poor and which are good. The goal is to make predictions on new examples that are nearly as good as the predictions made by the best predictor. We study an effective multiplicative-weights prediction method that associates a positive real weight with each predictor and multiplicatively decreases the weights associated with predictors when they make poor predictions. The model in this section is online (see Chapter 27): at each step, we do not know anything about the future examples. In addition, we are able to make predictions even in the presence of adversarial experts, who are collaborating against us, a situation that actually happens in game-playing settings.

Finally, Section 33.3 introduces gradient descent, a powerful optimization technique used to find parameter settings in machine-learning models. Gradient descent also has many applications outside of machine learning. Intuitively, gradient descent finds the value that produces a local minimum for a function by "walking downhill." In a learning application, a "downhill step" is a step that adjusts hypothesis parameters so that the hypothesis does better on the given set of labeled examples.

This chapter makes extensive use of vectors. In contrast to the rest of the book, vector names in this chapter appear in boldface, such as $\mathbf{x}$, to more clearly delineate which quantities are vectors. Components of vectors do not appear in boldface, so if vector $\mathbf{x}$ has d dimensions, we might write $\mathbf{x} = (x_1, x_2, \ldots, x_d)$.

33.1 Clustering

Suppose that you have a large number of data points (examples), and you wish to group them into classes based on how similar they are to each other. For example, each data point might represent a celestial star, giving its temperature, size, and spectral characteristics. Or, each data point might represent a fragment of recorded speech. Grouping these speech fragments appropriately might reveal the set of accents of the fragments. Once a grouping of the training data points is found, new data can be placed into an appropriate group, facilitating star-type recognition or speech recognition.

These situations, along with many others, fall under the umbrella of clustering. The input to a ***clustering*** problem is a set of n examples (objects) and an integer k, with the goal of dividing the examples into at most k disjoint clusters such that

the examples in each cluster are similar to each other. The clustering problem has several variations. For example, the integer k might not be given, but instead arises out of the clustering procedure. In this section we presume that k is given.

Feature vectors and similarity

Let's formally define the clustering problem. The input is a set of n ***examples***. Each example has a set of ***attributes*** in common with all other examples, though the attribute values may vary among examples. For example, the clustering problem shown in Figure 33.1 clusters $n = 49$ examples—48 state capitals plus the District of Columbia—into $k = 4$ clusters. Each example has two attributes: the latitude and longitude of the capital. In a given clustering problem, each example has d attributes, with an example $\mathbf{x}$ specified by a d-dimensional ***feature vector***

$$\mathbf{x} = (x_1, x_2, \ldots, x_d) .$$

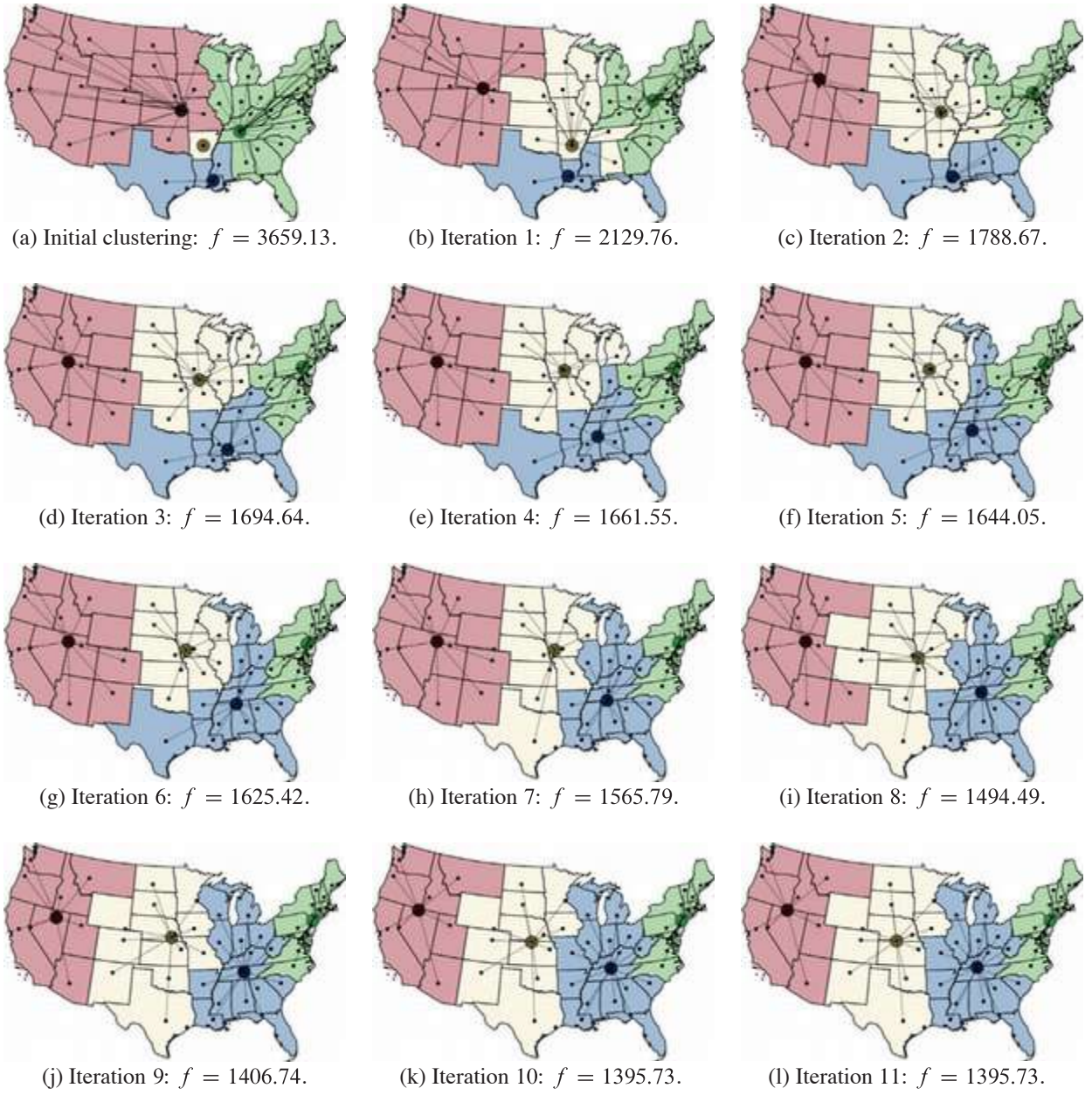
Here, x_a for $a = 1, 2, \ldots, d$ is a real number giving the value of attribute a for example $\mathbf{x}$. We call $\mathbf{x}$ the ***point*** in $\mathbb{R}^d$ representing the example. For the example in Figure 33.1, each capital $\mathbf{x}$ has its latitude in x_1 and its longitude in x_2.

In order to cluster similar points together, we need to define similarity. Instead, let's define the opposite: the ***dissimilarity*** $\Delta(\mathbf{x}, \mathbf{y})$ of points $\mathbf{x}$ and $\mathbf{y}$ is the squared Euclidean distance between them:

$$\begin{aligned} \Delta(\mathbf{x}, \mathbf{y}) &= \|\mathbf{x} - \mathbf{y}\|^2 \\ &= \sum_{a=1}^{d} (x_a - y_a)^2 . \end{aligned} \tag{33.1}$$

Of course, for $\Delta(\mathbf{x}, \mathbf{y})$ to be well defined, all attribute values must be present. If any are missing, then you might just ignore that example, or you could fill in a missing attribute value with the median value for that attribute.

The attribute values are often "messy" in other ways, so that some "data cleaning" is necessary before the clustering algorithm is run. For example, the scale of attribute values can vary widely across attributes. In the example of Figure 33.1, the scales of the two attributes vary by a factor of 2, since latitude ranges from -90 to $+90$ degrees but longitude ranges from -180 to $+180$ degrees. You can imagine other scenarios where the differences in scales are even greater. If the examples contain information about students, one attribute might be grade-point average but another might be family income. Therefore, the attribute values are usually scaled or normalized, so that no single attribute can dominate the others when computing dissimilarities. One way to do so is by scaling attribute values with a linear transform so that the minimum value becomes 0 and the maximum value becomes 1. If the attribute values are binary values, then no scaling may be needed. Another



(a) Initial clustering: $f = 3659.13$. (b) Iteration 1: $f = 2129.76$. (c) Iteration 2: $f = 1788.67$.

(d) Iteration 3: $f = 1694.64$. (e) Iteration 4: $f = 1661.55$. (f) Iteration 5: $f = 1644.05$.

(g) Iteration 6: $f = 1625.42$. (h) Iteration 7: $f = 1565.79$. (i) Iteration 8: $f = 1494.49$.

(j) Iteration 9: $f = 1406.74$. (k) Iteration 10: $f = 1395.73$. (l) Iteration 11: $f = 1395.73$.

Figure 33.1 The iterations of Lloyd's procedure when clustering the capitals of the lower 48 states and the District of Columbia into $k = 4$ clusters. Each capital has two attributes: latitude and longitude. Each iteration reduces the value f, measuring the sum of squares of distances of all capitals to their cluster centers, until the value of f does not change. **(a)** The initial four clusters, with the capitals of Arkansas, Kansas, Louisiana, and Tennessee chosen as centers. **(b)–(k)** Iterations of Lloyd's procedure. **(l)** The 11th iteration results in the same value of f as the 10th iteration in part (k), and so the procedure terminates.

option is scaling so that the values for each attribute have mean 0 and unit variance. Sometimes it makes sense to choose the same scaling rule for several related attributes (for example, if they are lengths measured to the same scale).

Also, the choice of dissimilarity measure is somewhat arbitrary. The use of the sum of squared differences as in equation (33.1) is not required, but it is a conventional choice and mathematically convenient. For the example of Figure 33.1, you might use the actual distance between capitals rather than equation (33.1).

Clusterings

With the notion of similarity (actually, *dis*similarity) defined, let's see how to define clusters of similar points. Let S denote the given set of n points in $\mathbb{R}^d$. In some applications the points are not necessarily distinct, so that S is a multiset rather than a set.

Because the goal is to create k clusters, we define a ***k-clustering*** of S as a decomposition of S into a sequence $\langle S^{(1)}, S^{(2)}, \ldots, S^{(k)} \rangle$ of k disjoint subsets, or ***clusters***, so that

$$S = S^{(1)} \cup S^{(2)} \cup \cdots \cup S^{(k)} .$$

A cluster may be empty, for example if $k > 1$ but all of the points in S have the same attribute values.

There are many ways to define a k-clustering of S and many ways to evaluate the quality of a given k-clustering. We consider here only k-clusterings of S that are defined by a sequence C of k ***centers***

$$C = \langle \mathbf{c^{(1)}}, \mathbf{c^{(2)}}, \ldots, \mathbf{c^{(k)}} \rangle ,$$

where each center is a point in $\mathbb{R}^d$, and the ***nearest-center rule*** says that a point $\mathbf{x}$ may belong to cluster $S^{(\ell)}$ if the center of no other cluster is closer to $\mathbf{x}$ than the center $\mathbf{c^{(\ell)}}$ of $S^{(\ell)}$:

$$\mathbf{x} \in S^{(\ell)} \text{ only if } \Delta(\mathbf{x}, \mathbf{c^{(\ell)}}) = \min \{\Delta(\mathbf{x}, \mathbf{c^{(j)}}) : 1 \le j \le k\} .$$

A center can be anywhere, and not necessarily a point in S.

Ties are possible and must be broken so that each point lies in exactly one cluster. In general, ties may be broken arbitrarily, although we'll need the property that we never change which cluster a point $\mathbf{x}$ is assigned to unless the distance from $\mathbf{x}$ to its new cluster center is *strictly smaller* than the distance from $\mathbf{x}$ to its old cluster center. That is, if the current cluster has a center that is one of the closest cluster centers to $\mathbf{x}$, then don't change which cluster $\mathbf{x}$ is assigned to.

The ***k-means problem*** is then the following: given a set S of n points and a positive integer k, find a sequence $C = \langle \mathbf{c^{(1)}}, \mathbf{c^{(2)}}, \ldots, \mathbf{c^{(k)}} \rangle$ of k center points